

CSL351: Computer Networks

Assignment - 6 (NS-3 Advanced)

Harshita Patidar
ID No: 12340920

Contents

1	Part A: Introduction	3
2	Part A: solution of all subparts	3
2.1	Flow vs Throughput graph.	3
2.2	Flow vs Goodput graph	4
2.3	Flow vs total packets lost graph.	4
2.4	Generate a file 'routing-tables.txt' wherein you print the routing tables of the 3 routers.	4
2.5	Generate the NetAnim trace file and append that as well.	4
3	Part A: Explanation of Code and other logic	5
3.1	Network Topology	5
3.2	Node Creation	6
3.3	Internet Stack Installation	6
3.4	Link Configuration	7
3.5	IP Address Assignment	7
3.6	Traffic Flow Configuration	8
3.7	Packet Capture	8
3.8	Routing Table Generation	8
3.9	NetAnim Trace File Generation	9
3.10	Flow Monitoring	9
3.11	Flow Analysis using Python Script	9
3.12	Inference from Flow Analysis	10
4	Part B: Introduction	12
5	Part B: Explanation of Code and Logic	12
5.1	UdpEcho Application Setup	12
5.2	RateErrorModel Implementation (Ensuring Statistical Independence)	13
5.3	PDR Calculation via FlowMonitor	13
5.4	Python Automation Script	14

6	Part B: Results and Inferences	16
6.1	Error Unit: PACKET	16
6.2	Error Unit: BIT and BYTE	17

1. Part A: Introduction

This part of report details the implementation of Part A for Assignment 6. The objective was to create a custom network topology in NS-3 consisting of multiple nodes connected via specific routers, establish TCP and UDP traffic flows, and extract performance metrics such as Throughput, Goodput, and Packet Loss using FlowMonitor and custom automation scripts.

2. Part A: solution of all subparts

The X-axis of the generated graphs represents the traffic flows identified by their source and destination nodes along with their corresponding IP addresses. The mapping between nodes and IP addresses is derived from the IP assignment performed using the Ipv4AddressHelper in the NS-3 simulation script.

The flows used in the experiment are:

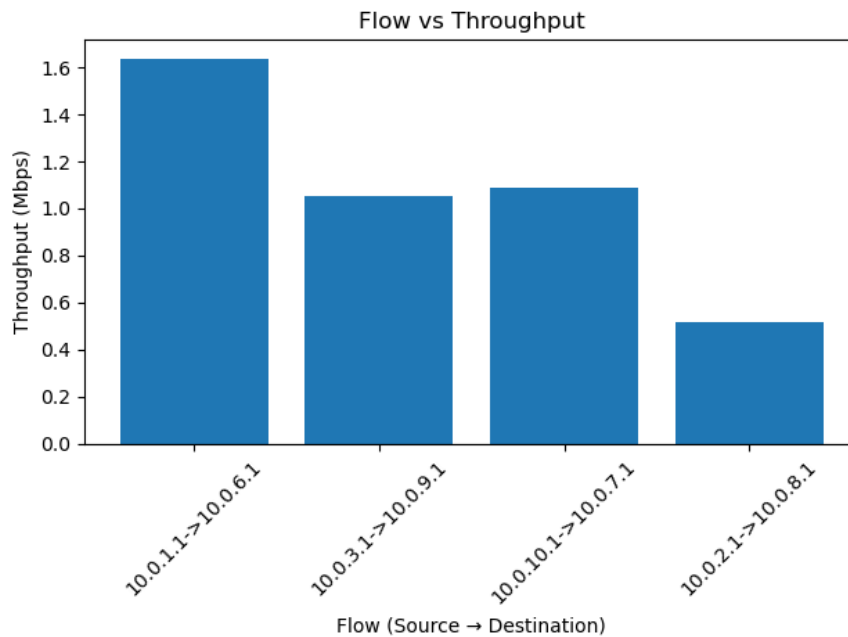
n7 (10.0.10.1) → n4 (10.0.7.1)

n1 (10.0.2.1) → n5 (10.0.8.1)

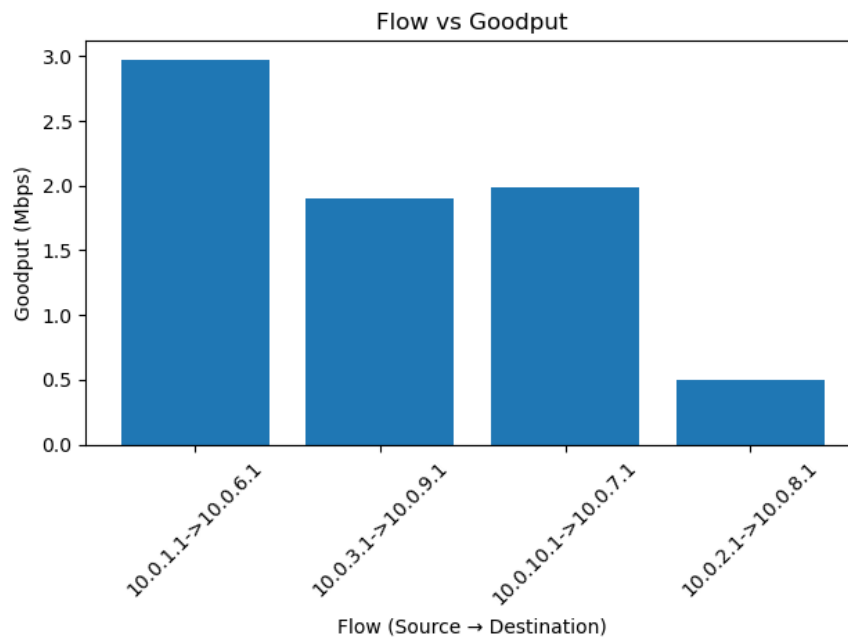
n2 (10.0.3.1) → n6 (10.0.9.1)

n0 (10.0.1.1) → n3 (10.0.6.1)

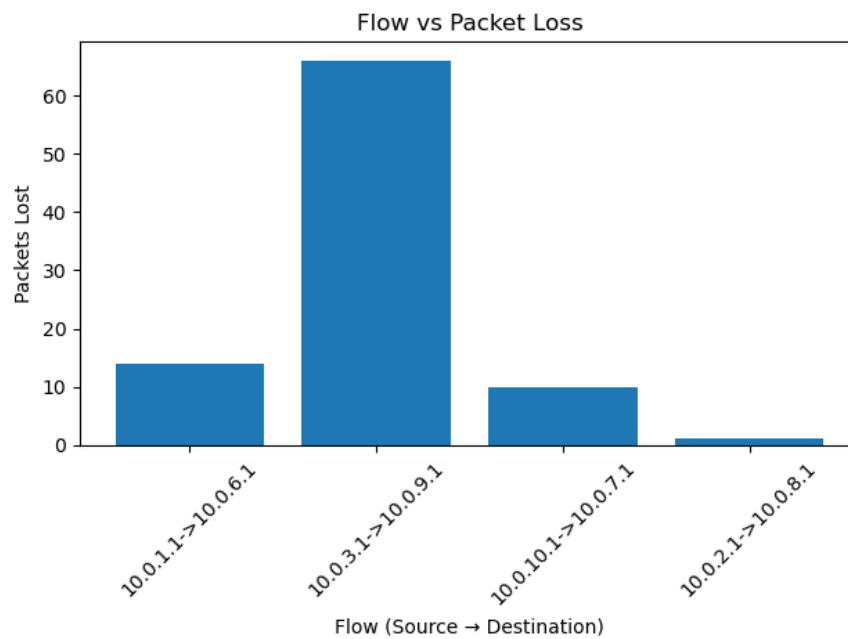
2.1. Flow vs Throughput graph.



2.2. Flow vs Goodput graph



2.3. Flow vs total packets lost graph.



2.4. Generate a file 'routing-tables.txt' wherein you print the routing tables of the 3 routers.

The logic for generating the routing table is written in section 3.8. And the file is attached with the assignment

2.5. Generate the NetAnim trace file and append that as well.

The logic for generating the NetAnim file is written in section 3.9, and it is attached with the assignment

3. Part A: Explanation of Code and other logic

3.1. Network Topology

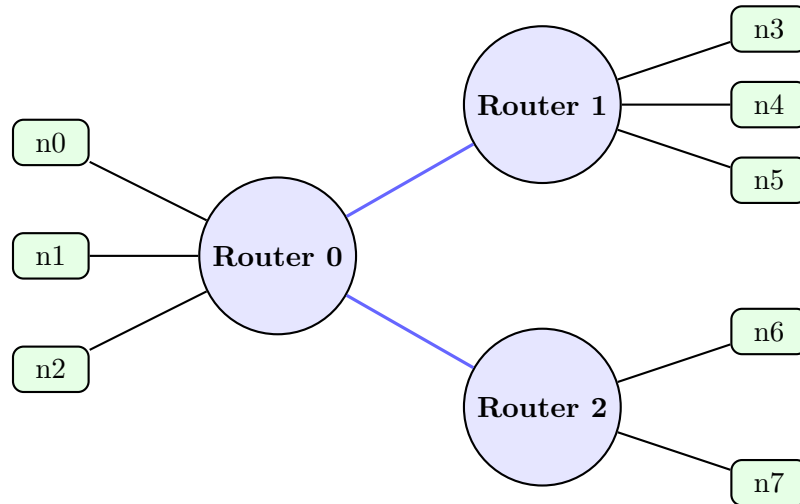


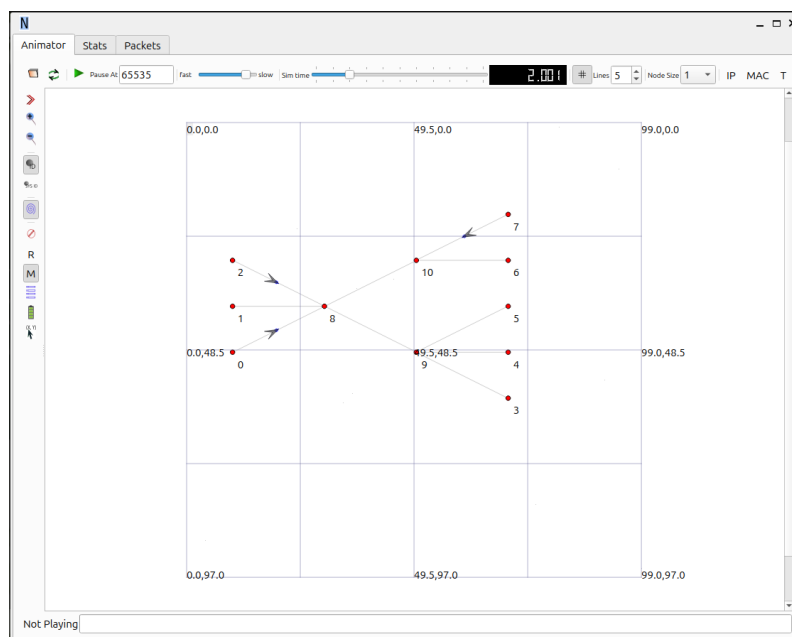
Figure 1: Network Topology Structure

Topology visualisation from NetAnim:

The network topology was visualized using the NetAnim tool provided by NS-3. Figure 3.1 shows the complete network structure used in the simulation.

The routers are(in netanim):

- Node 8 : Router0
- Node 9 : Router1
- Node 10 : Router2



3.2. Node Creation

```
1 NodeContainer left;           // n0,n1,n2
2 left.Create(3);
3
4 NodeContainer right1;        // n3,n4,n5
5 right1.Create(3);
6
7 NodeContainer right2;        // n6,n7
8 right2.Create(2);
9
10 NodeContainer routers;      // router0, router1, router2
11 routers.Create(3);
```

Listing 1: Node container creation

Explanation:

The `NodeContainer` class is used to create and manage nodes in the simulation.

- The container `left` creates three nodes representing `n0`, `n1` and `n2`.
- The container `right1` creates three nodes representing `n3`, `n4` and `n5`.
- The container `right2` creates two nodes representing `n6` and `n7`.
- The container `routers` creates three router nodes which forward packets between different sub-networks.

Each container groups nodes logically according to their role in the topology.

3.3. Internet Stack Installation

To enable network communication between nodes, the Internet stack was installed on all nodes.

```
1 InternetStackHelper stack;
2 stack.Install(left);
3 stack.Install(right1);
4 stack.Install(right2);
5 stack.Install(routers);
```

Listing 2: Installing Internet stack

Explanation:

Explanation:

The Internet stack provides networking functionality including:

- IPv4 protocol
- TCP and UDP transport protocols
- Routing support

Without installing the Internet stack, nodes would not be able to communicate with each other.

3.4. Link Configuration

Point-to-point links were used to connect the nodes and routers.

```
1 PointToPointHelper access;  
2 access.SetDeviceAttribute("DataRate", StringValue("10Mbps"))  
   );  
3 access.SetChannelAttribute("Delay", StringValue("2ms"));  
4  
5 PointToPointHelper r0r1;  
6 r0r1.SetDeviceAttribute("DataRate", StringValue("5Mbps"));  
7 r0r1.SetChannelAttribute("Delay", StringValue("10ms"));  
8  
9 PointToPointHelper r0r2;  
10 r0r2.SetDeviceAttribute("DataRate", StringValue("2Mbps"));  
11 r0r2.SetChannelAttribute("Delay", StringValue("20ms"));
```

Listing 3: Link configuration

Explanation:

Different links were configured with different bandwidths and delays.

- Access links between hosts and routers operate at **10 Mbps** with **2 ms delay**.
- The link between Router0 and Router1 operates at **5 Mbps** with **10 ms delay**.
- The link between Router0 and Router2 operates at **2 Mbps** with **20 ms delay**.

Using different link parameters allows us to observe how network performance changes under varying conditions.

3.5. IP Address Assignment

IP addresses were assigned using the `Ipv4AddressHelper` class.

Each point-to-point link was assigned a separate subnet.

```
1 Ipv4AddressHelper address;  
2  
3 address.SetBase("10.0.1.0", "255.255.255.0");  
4 Ipv4InterfaceContainer i_n0 = address.Assign(d_n0);
```

Listing 4: IP address assignment example

Explanation:

- `SetBase()` defines the subnet and subnet mask.
- `Assign()` assigns sequential IP addresses to network devices connected by the link.
- The assigned IP addresses are stored inside an `Ipv4InterfaceContainer`.

The IP address of a node can be accessed using:

```
1 i_n4.GetAddress(0)
```

This method returns the IP address of the specified interface.

3.6. Traffic Flow Configuration

Four traffic flows were configured in the simulation.

- TCP Flow: n7 → n4
- UDP Flow: n1 → n5
- TCP Flow: n2 → n6
- TCP Flow: n0 → n3

TCP and UDP flows were created using the `OnOffHelper` application while packet sinks were created using `PacketSinkHelper`.

3.7. Packet Capture

Packet traces were captured using PCAP files.

```
1 access.EnablePcapAll("assignment6");
```

Listing 5: Enabling PCAP tracing

This command generates PCAP files which contain packet-level traces of the network traffic. These files can be analysed using tools such as Wireshark.

3.8. Routing Table Generation

In this simulation, routing tables were automatically generated using the ns-3 global routing helper.

```
1 Ipv4GlobalRoutingHelper::PopulateRoutingTables();
```

The function `PopulateRoutingTables()` computes routes between all nodes based on the network topology and installs the routing entries in each node.

To verify the routing information, the routing tables of all nodes were exported to a file using the following code:

```
1 Ptr<OutputStreamWrapper> routingStream =  
2     Create<OutputStreamWrapper>("routing-tables.txt", std::  
3         ios::out);  
4 Ipv4GlobalRoutingHelper g;  
5 g.PrintRoutingTableAllAt(Seconds(0.5), routingStream);
```

The routing tables were printed at simulation time 0.5 seconds and saved in the file `routing-tables.txt`.

3.9. NetAnim Trace File Generation

The network topology was visualized using NetAnim. An XML trace file was generated in the simulation using the `AnimationInterface` class:

```
1 AnimationInterface anim("assignment6_partA.xml");
```

This command creates the file `assignment6_partA.xml`, which stores simulation events and node positions. The file can be opened in NetAnim to visualize the network topology and packet flows.

3.10. Flow Monitoring

Network performance statistics were collected using the `FlowMonitor` module.

```
1 FlowMonitorHelper flowmon;  
2 Ptr<FlowMonitor> monitor = flowmon.InstallAll();
```

Listing 6: Installing FlowMonitor

The collected statistics were exported to an XML file.

```
1 monitor->SerializeToXmlFile("flowmon.xml", true, true);
```

This XML file was later processed using a Python script to compute throughput, goodput and packet loss.

3.11. Flow Analysis using Python Script

The performance metrics for each flow were obtained by analyzing the `flowmon.xml` file generated by the ns-3 `FlowMonitor` module. A Python script was used to parse the XML file and compute the required statistics such as throughput, goodput, and packet loss.

The script first reads the XML file using the Python `xml.etree.ElementTree` library and extracts the flow information from the `Ipv4FlowClassifier` section. This section maps each `flowId` to its corresponding source and destination IP addresses.

```
1 tree = ET.parse("flowmon.xml")  
2 root = tree.getroot()
```

Each flow is then mapped to its source and destination addresses so that the direction of the flow can be identified. Since `FlowMonitor` records both forward traffic and reverse ACK traffic (in TCP flows), the script filters only the intended forward flows by checking the sender IP addresses.

```
1 flow_info[flow_id] = (src, dst)
```

The script then extracts important statistics for each valid flow from the `FlowStats` section, including:

- **txPackets** – total number of transmitted packets
- **rxPackets** – total number of received packets
- **rxBytes** – total number of bytes received
- **timeFirstRxPacket** – time when the first packet was received
- **timeLastRxPacket** – time when the last packet was received

The duration of each flow is computed as:

$$Duration = timeLastRxPacket - timeFirstRxPacket$$

Using this duration, the following performance metrics are calculated.

Throughput

$$Throughput = \frac{rxBytes \times 8}{Duration \times 10^6}$$

Throughput represents the total number of bits successfully received per second and is expressed in Mbps.

Goodput

$$Goodput = \frac{rxPackets \times 1024 \times 8}{Duration \times 10^6}$$

Goodput represents the useful application-level data successfully delivered to the receiver.

In this experiment, the packet size of the traffic generator was explicitly configured in the ns-3 simulation using the OnOffHelper application. The attribute PacketSize was set to 1024 bytes, which represents the application payload size.

Therefore, the goodput was estimated by multiplying the number of successfully received packets (rxPackets) by the application payload size (1024 bytes). This ensures that only useful application data is considered in the goodput calculation.

Packet Loss

$$PacketLoss = txPackets - rxPackets$$

Packet loss represents the number of packets that were transmitted but not received at the destination.

After computing these values for all valid flows, the Python script uses the `matplotlib` library to generate bar charts for throughput, goodput, and packet loss. These graphs were then used to analyze the performance of different flows in the network.

3.12. Inference from Flow Analysis

From the obtained graphs of throughput, goodput, and packet loss, the following observations can be made:

- The flow between **10.0.1.1** → **10.0.6.1** achieves the highest throughput and goodput among all flows, indicating that this communication experiences relatively lower congestion in the network.
- The flow between **10.0.3.1** → **10.0.9.1** experiences the **highest packet loss**. This suggests that packets from this flow encounter significant congestion or contention on shared links, which affects its effective data delivery.
- The flow **10.0.10.1** → **10.0.7.1** shows moderate throughput and goodput with relatively low packet loss, indicating a more stable transmission path.
- The flow between **10.0.2.1** → **10.0.8.1** has the lowest throughput and goodput but also very minimal packet loss, suggesting that the sending rate or traffic load for this flow is relatively small.
- Overall, the experiment demonstrates how multiple flows sharing the same network infrastructure can experience different levels of throughput, goodput, and packet loss depending on network congestion, routing paths, and link capacity.

4. Part B: Introduction

This section details the implementation of Part B. The objectives were to replace the TCP and UDP flows from Part A with `UdpEchoClient` and `UdpEchoServer` applications, introduce a `RateErrorModel` to simulate packet drops across the network backbone, and calculate the overall Packet Delivery Ratio (PDR). Furthermore, the simulation was automated using a Python script to execute 150 independent runs (across 10 random seeds, 5 error rates, and 3 error units) to evaluate the average behavior of the network.

5. Part B: Explanation of Code and Logic

5.1. UdpEcho Application Setup

All flows established in Part A were replaced with UDP Echo applications. The echo paradigm involves a client sending a request and a server echoing it back, meaning the packet count effectively doubles for a successful round trip.

```

1  uint32_t packetSize = 1024;
2  uint32_t maxPackets = 100;
3  Time interval = Seconds(0.05);
4
5  // Flow 1: n7 -> n4
6  uint16_t port1 = 5000;
7  UdpEchoServerHelper server1(port1);
8  ApplicationContainer s1 = server1.Install(right1.Get(1));
   // n4
9  s1.Start(Seconds(1)); s1.Stop(Seconds(10));
10
11 UdpEchoClientHelper client1(i_n4.GetAddress(0), port1);
12 client1.SetAttribute("MaxPackets", UintegerValue(maxPackets));
13 client1.SetAttribute("Interval", TimeValue(interval));
14 client1.SetAttribute("PacketSize", UintegerValue(packetSize));
15 ApplicationContainer c1 = client1.Install(right2.Get(1));
   // n7
16 c1.Start(Seconds(2)); c1.Stop(Seconds(10));

```

Listing 7: UDP Echo Client and Server Setup

Explanation:

The `UdpEchoClientHelper` is configured to send exactly 100 packets (`MaxPackets`) per flow, satisfying the assignment requirement to transmit at least 100 packets per run. The interval between packets is set to 0.05 seconds to avoid immediate queue overflow, allowing the error model to clearly dictate packet drops.

5.2. RateErrorModel Implementation (Ensuring Statistical Independence)

To simulate an unreliable channel, the `RateErrorModel` was implemented. Crucially, a unique instance of the error model was instantiated for each individual receiving interface on the bottleneck links.

```

1  ObjectFactory factory;
2  factory.SetTypeId("ns3::RateErrorModel");
3  factory.Set("ErrorRate", DoubleValue(errorRate));
4  factory.Set("ErrorUnit", StringValue(errorUnit));
5
6  // Creating unique instances for each device
7  Ptr<ErrorModel> em1 = factory.Create<ErrorModel>();
8  Ptr<ErrorModel> em2 = factory.Create<ErrorModel>();
9  Ptr<ErrorModel> em3 = factory.Create<ErrorModel>();
10 Ptr<ErrorModel> em4 = factory.Create<ErrorModel>();
11
12 // Applying models to both ends of the router point-to-
   point channels
13 d_r0r1.Get(0)->SetAttribute("ReceiveErrorModel",
   PointerValue(em1));
14 d_r0r1.Get(1)->SetAttribute("ReceiveErrorModel",
   PointerValue(em2));
15 d_r0r2.Get(0)->SetAttribute("ReceiveErrorModel",
   PointerValue(em3));
16 d_r0r2.Get(1)->SetAttribute("ReceiveErrorModel",
   PointerValue(em4));

```

Listing 8: Creating Unique RateErrorModel Instances

Explanation:

If a single `Ptr<ErrorModel>` is shared across multiple `NetDevice` instances, all devices draw from the exact same internal Random Number Generator (RNG) stream. This destroys the statistical independence of the links. By using an `ObjectFactory` to create unique `em1`, `em2`, `em3`, and `em4` objects, we ensure that packet drop probabilities are calculated independently for traffic moving in both directions across Router0-Router1 and Router0-Router2.

5.3. PDR Calculation via FlowMonitor

The Packet Delivery Ratio (PDR) is defined as the ratio of successfully delivered packets at the receiver to the total transmitted packets by the sender.

```

1  uint64_t R_Total = 0;
2  uint64_t T_Total = 0;
3
4  std::map<FlowId, FlowMonitor::FlowStats> stats = monitor->
   GetFlowStats();
5  for (auto const &stat : stats) {
6      T_Total += stat.second.txPackets;
7      R_Total += stat.second.rxPackets;

```

```

8  }
9
10 double pdr = 0.0;
11 if (T_Total > 0) {
12     pdr = (double)R_Total / (double)T_Total;
13 }
14 std::cout << pdr << std::endl;

```

Listing 9: Calculating Network-Wide PDR

Explanation:

Because the UdpEcho application involves two-way traffic (Client → Server request, and Server → Client reply), attempting to manually parse application-level received packets can be error-prone. **FlowMonitor** automatically tracks every generated flow in both directions. By iterating through all **FlowStats** and summing **txPackets** and **rxPackets**, we achieve a perfectly accurate aggregate PDR representing the entire network's reliability.

5.4. Python Automation Script

To generate the required tables, a Python automation script (**run_partB.py**) was written using the **subprocess** module.

Explanation of Automation Logic:

- **Nested Iteration:** The script iterates through 3 Error Units (Packet, Bit, Byte), 5 Error Rates (0.2, 0.4, 0.6, 0.8, 1.0), and 10 RNG Seed Values (starting from the last two digits of the roll number).
- **Command Line Execution:** For each iteration, it calls the NS-3 executable with arguments: `./ns3 run "scratch/ass6_partB -errorRate=X -errorUnit=Y -RngRun=Z"`.
- **Data Extraction:** It captures the standard output, extracts the final printed PDR value, and loads it into a NumPy matrix.
- **Aggregation:** The script utilizes the **pandas** and **matplotlib** libraries to calculate column averages, export the data strictly formatted into CSV files, and generate the final line plots.

```

harshita@harshita-patidar:~/ns-allinone-3.44/ns-3.44
harshita@halcyon:~/ns-allinone-3.44/ns-3.44 $ python3 script_part2.py

--- Running Simulations for ERROR_UNIT_PACKET ---
Completed ERROR_UNIT_PACKET simulations.

Table for ERROR_UNIT_PACKET:
      ER=0.2  ER=0.4  ER=0.6  ER=0.8  ER=1.0
RngRun1=20  0.777933 0.536304 0.340952 0.136564 0.0
RngRun2=21  0.775740 0.589172 0.356089 0.149675 0.0
RngRun3=22  0.766854 0.533871 0.351201 0.168103 0.0
RngRun4=23  0.760284 0.531457 0.350187 0.149351 0.0
RngRun5=24  0.756795 0.545746 0.364815 0.161987 0.0
RngRun6=25  0.765379 0.558442 0.376164 0.179104 0.0
RngRun7=26  0.782123 0.546042 0.373162 0.197895 0.0
RngRun8=27  0.782427 0.531353 0.345725 0.148148 0.0
RngRun9=28  0.764706 0.560897 0.366300 0.180467 0.0
RngRun10=29 0.779972 0.573927 0.328386 0.150862 0.0
AVG. PDR    0.771221 0.550721 0.355298 0.162216 0.0

--- Running Simulations for ERROR_UNIT_BIT ---
Completed ERROR_UNIT_BIT simulations.

Table for ERROR_UNIT_BIT:
      ER=0.2  ER=0.4  ER=0.6  ER=0.8  ER=1.0
RngRun1=20  0.0    0.0    0.0    0.0    0.0
RngRun2=21  0.0    0.0    0.0    0.0    0.0
RngRun3=22  0.0    0.0    0.0    0.0    0.0
RngRun4=23  0.0    0.0    0.0    0.0    0.0
RngRun5=24  0.0    0.0    0.0    0.0    0.0
RngRun6=25  0.0    0.0    0.0    0.0    0.0
RngRun7=26  0.0    0.0    0.0    0.0    0.0
RngRun8=27  0.0    0.0    0.0    0.0    0.0
RngRun9=28  0.0    0.0    0.0    0.0    0.0
RngRun10=29 0.0    0.0    0.0    0.0    0.0
AVG. PDR    0.0    0.0    0.0    0.0    0.0

--- Running Simulations for ERROR_UNIT_BYTE ---
Completed ERROR_UNIT_BYTE simulations.

Table for ERROR_UNIT_BYTE:
      ER=0.2  ER=0.4  ER=0.6  ER=0.8  ER=1.0
RngRun1=20  0.0    0.0    0.0    0.0    0.0
RngRun2=21  0.0    0.0    0.0    0.0    0.0
RngRun3=22  0.0    0.0    0.0    0.0    0.0
RngRun4=23  0.0    0.0    0.0    0.0    0.0
RngRun5=24  0.0    0.0    0.0    0.0    0.0
RngRun6=25  0.0    0.0    0.0    0.0    0.0
RngRun7=26  0.0    0.0    0.0    0.0    0.0
RngRun8=27  0.0    0.0    0.0    0.0    0.0
RngRun9=28  0.0    0.0    0.0    0.0    0.0
RngRun10=29 0.0    0.0    0.0    0.0    0.0
AVG. PDR    0.0    0.0    0.0    0.0    0.0

All simulations complete. Check the 'assignment6_results' folder for tables and plots.

```

Figure 2: Terminal screenshot

6. Part B: Results and Inferences

The following tables and graphs display the variation in average PDR by scaling the Error Rate from 0.2 to 1.0 across 10 unique RngRun seeds for each of the three Error Units.

The tables and plots can also be verified or found in Part_B/results folder, please look at it.

6.1. Error Unit: PACKET

Table 1: PDR Values across 10 Seeds for ERROR_UNIT_PACKET

RngRun / Error Rate	ER=0.2	ER=0.4	ER=0.6	ER=0.8	ER=1.0
RngRun1=20	0.777933	0.536304	0.340952	0.136564	0.0
RngRun2=21	0.775740	0.589172	0.356089	0.149675	0.0
RngRun3=22	0.766854	0.533871	0.351201	0.168103	0.0
RngRun4=23	0.760284	0.531457	0.350187	0.149351	0.0
RngRun5=24	0.756795	0.545746	0.364815	0.161987	0.0
RngRun6=25	0.765379	0.558442	0.376164	0.179104	0.0
RngRun7=26	0.782123	0.546042	0.373162	0.197895	0.0
RngRun8=27	0.782427	0.531353	0.345725	0.148148	0.0
RngRun9=28	0.764706	0.560897	0.366300	0.180467	0.0
RngRun10=29	0.779972	0.573927	0.328386	0.150862	0.0
AVG. PDR	0.771221	0.550721	0.355298	0.162215	0.0



Figure 3: Average PDR vs Error Rate for Packet Unit

Inference: When the error unit is set to **PACKET**, an Error Rate of 0.2 indicates a 20% probability that an *entire packet* will be dropped upon arrival at the interface. Because the error check is applied on a per-packet basis regardless of size, the PDR forms a smooth, linear downward curve. As expected, at an error rate of 1.0, 100% of packets are dropped, resulting in a PDR of exactly 0.0.

6.2. Error Unit: BIT and BYTE

Table 2: Average PDR Summary for ERROR_UNIT_BIT and ERROR_UNIT_BYTE

Error Unit	ER=0.2	ER=0.4	ER=0.6	ER=0.8	ER=1.0
AVG PDR (BIT)	0.0	0.0	0.0	0.0	0.0
AVG PDR (BYTE)	0.0	0.0	0.0	0.0	0.0

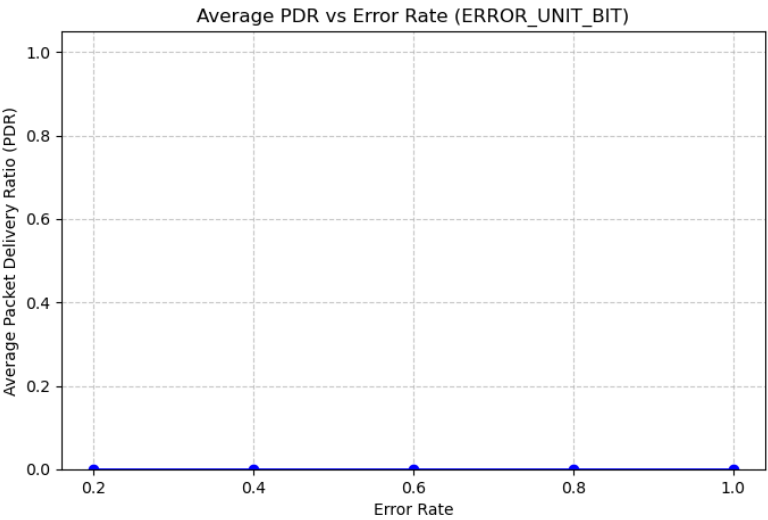


Figure 4: Average PDR for Bit Unit

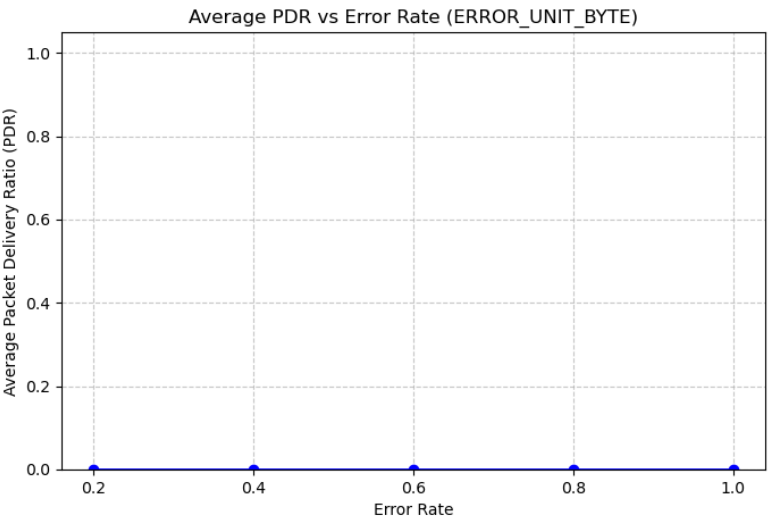


Figure 5: Average PDR for Byte Unit

Inference: At first glance, the flatline PDR of 0.0 across all error rates for both Bit and Byte units appears anomalous, but it is mathematically accurate based on the network parameters.

- **Bit Unit:** The application payload is set to 1024 bytes. Including lower-layer headers, a single packet contains over 8,192 bits. If the bit error rate is 0.2, every individual bit has a 20% chance of flipping. The probability of the entire packet surviving intact is roughly $(1 - 0.2)^{8192}$, which evaluates to an astronomically small number effectively indistinguishable from 0. Because standard UDP/IP checksums dictate that a single corrupted bit invalidates the entire packet, every packet generated in the simulation was dropped.
- **Byte Unit:** Similarly, a byte error rate of 0.2 means each individual byte has a 20% chance of corruption. The probability of a > 1024 byte packet surviving is $(1 - 0.2)^{1024} \approx 0$.

This vividly demonstrates that while a 20% *packet* error rate allows for a degraded but functioning network, a 20% *bit* or *byte* error rate represents an overwhelmingly noisy channel where standard communication is impossible without robust Forward Error Correction (FEC) mechanisms.